flowise-google sheet

Ref YouTube video
Date amended: 19th Feb, 2025
File: webhook Chatflow.json

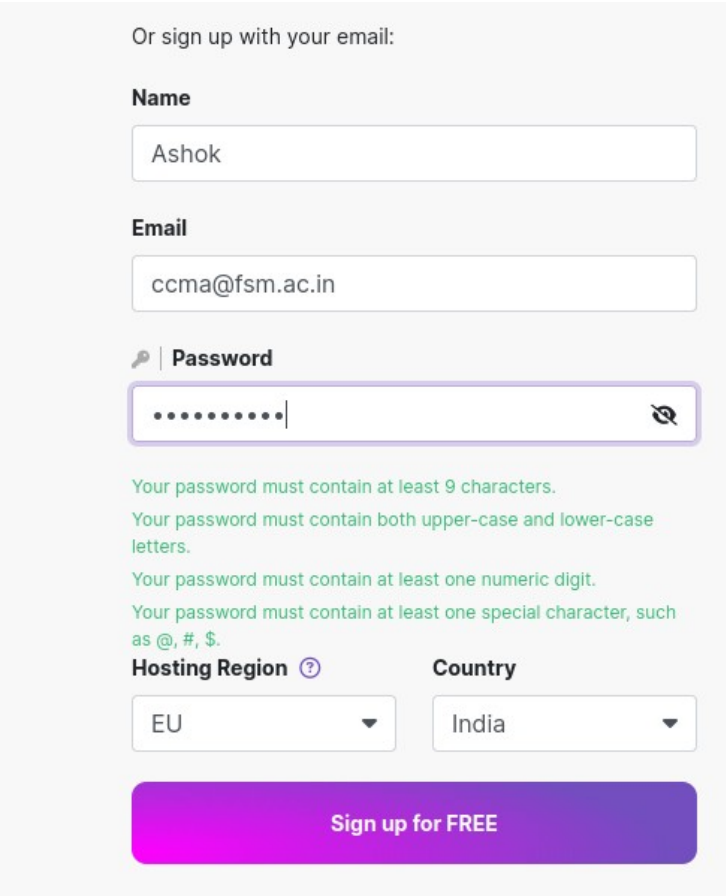
A. Create and test webhook first

Steps

1) Open your gdrive. And create a folder, say: *flowise*.
2) Rightclick within the folder and create a google spreadsheet, say flowise_webhook. Press ctrl+s to save it.
3) Rename *Sheet1* to nodes. This step is optional.

Figure 1: Gsheet on my gdrive

4) Write column heaaders: *Node*, *Category* and *Action*. These header-names (spellings, case) are very important.
5) Create an account in make.com. My passsword is: Gautam*89023



6. Loginto make.com. Select *Template*: Click *Add Webhook data to Google Sheet*

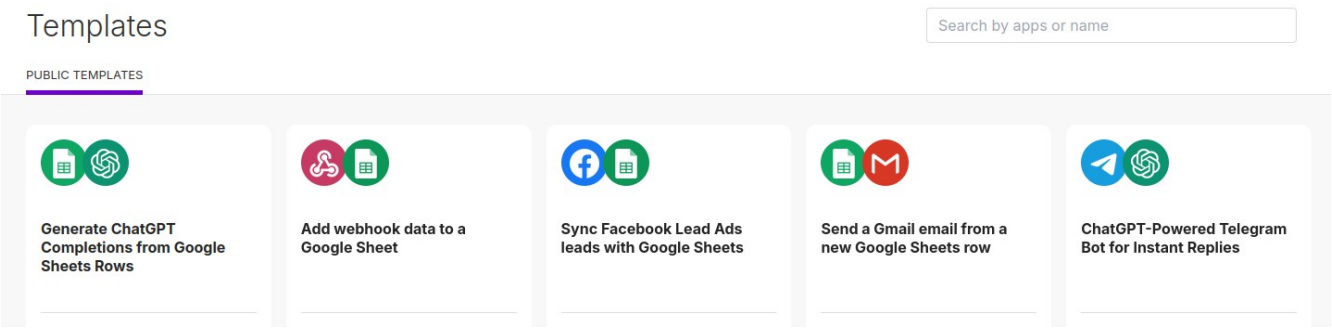


Figure 2: Select Add webhook data to gsheet.

What is a Webhook

Webhook is a software interface (or a *hook*) between two applications. To set up a webhook, the client (google sheet) gives a unique URL to the server API (flowise) and specifies which event it wants to know about. Once the webhook is set up, the client (gsheet) no longer needs to poll the server; the server will automatically send the relevant payload (using http POST) to the client's webhook URL (gsheet URL) when the specified event occurs.

Webhooks are often referred to as *reverse APIs* or *push APIs*, because they put the responsibility of communication on the server (flowise), rather than the client (gsheet). Instead of the client (gsheet) sending HTTP requests—asking for data until the server (flowise) responds—the server (flowise) itself sends the client a single HTTP POST request as soon as the data is available. Despite their nicknames, webhooks are not APIs; they work together. An application must have an API to use a webhook.

Read here in detail how make.com webhooks work.

7. Template Selected.

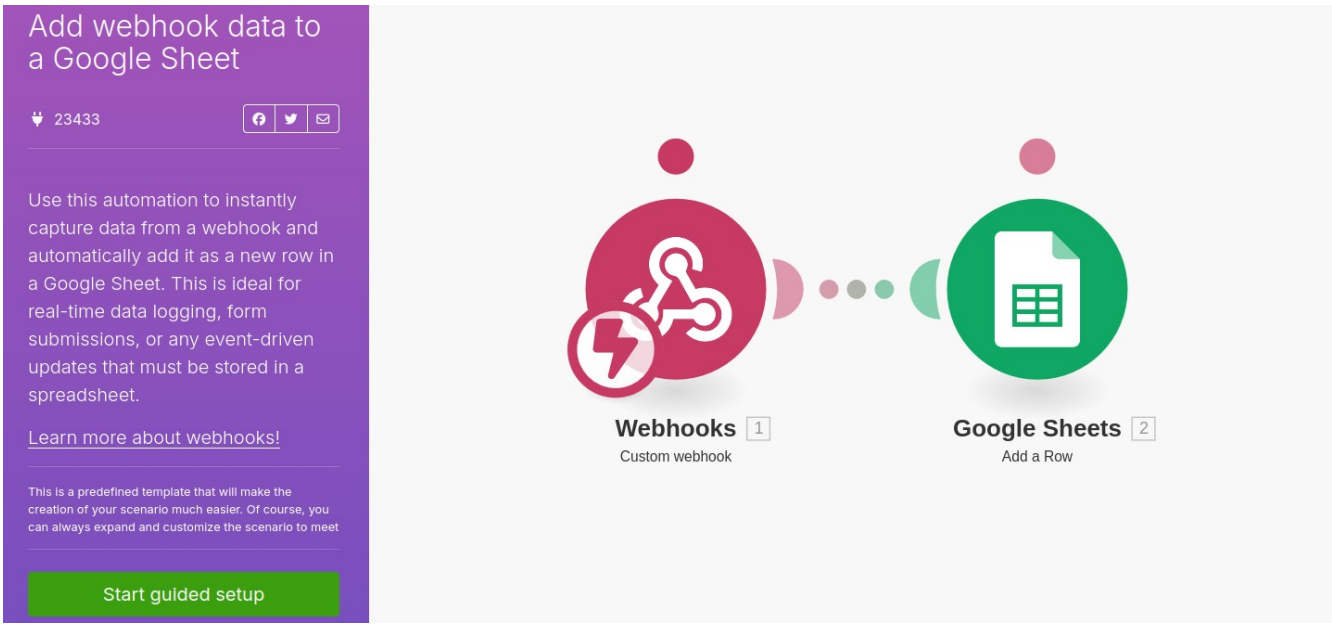


Figure 3: Select a webhook-gsheet template. This specific webhook is customised for gsheet.

8. Click on *Start guided setup* to setup template:

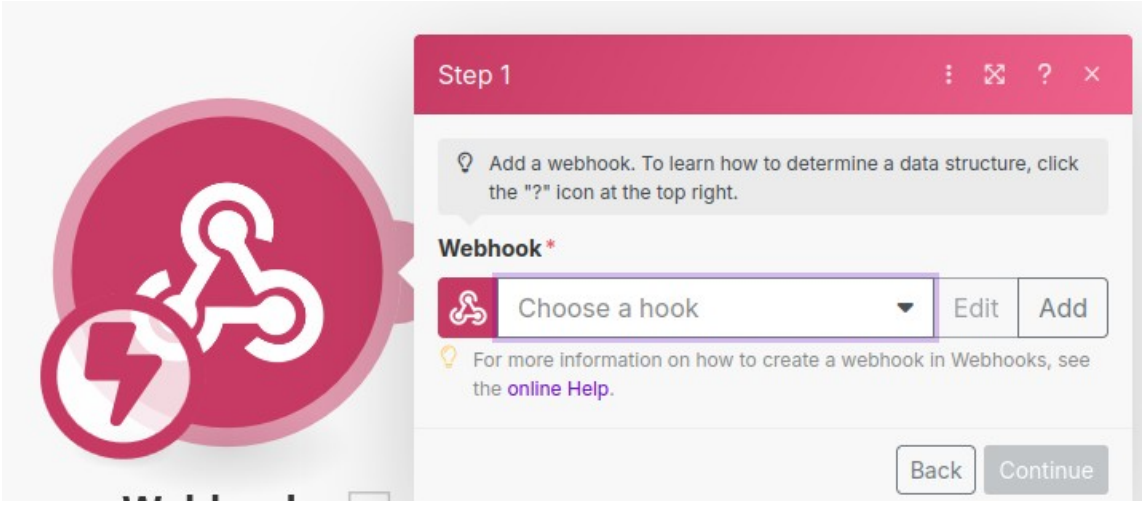


Figure 4: Step1 of the guided workflow. Choose a hook from the drop-down.

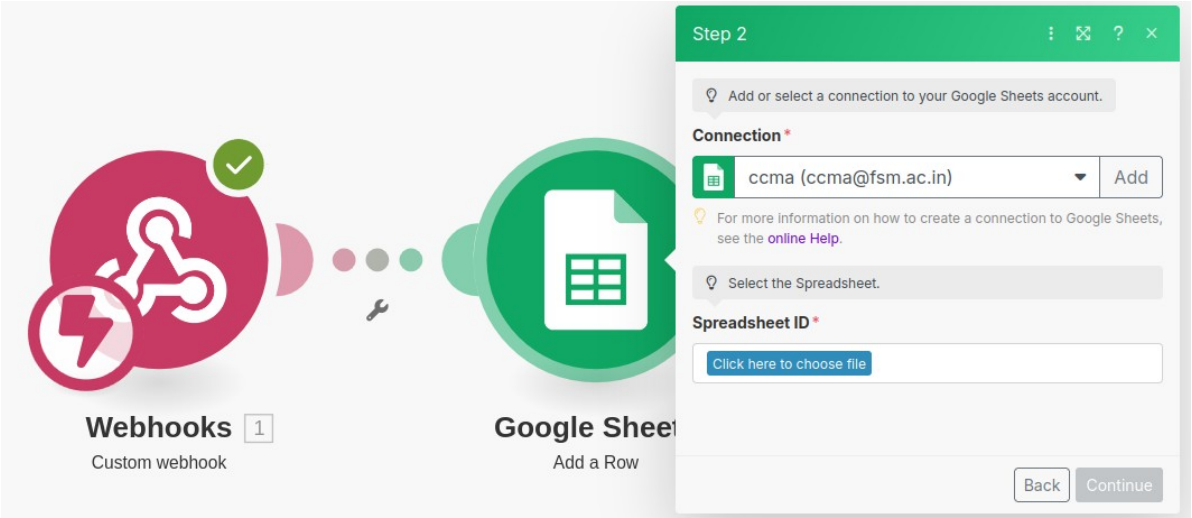


Figure 5: In Step 2, set up Google sheet. Select your gsheet by name.

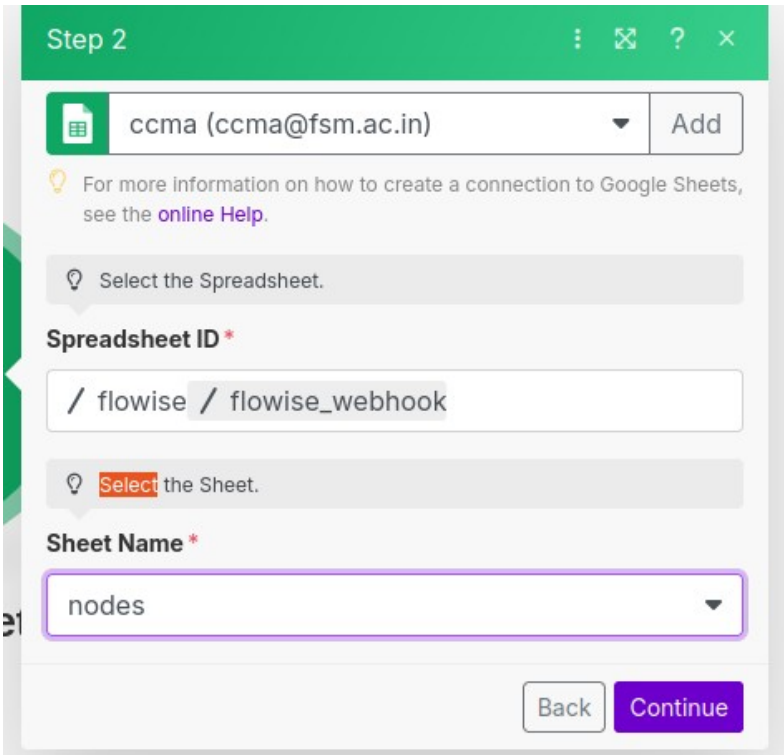


Figure 6: Select also the Sheet (by name) ie whether Sheet1 or Sheet2 etc.

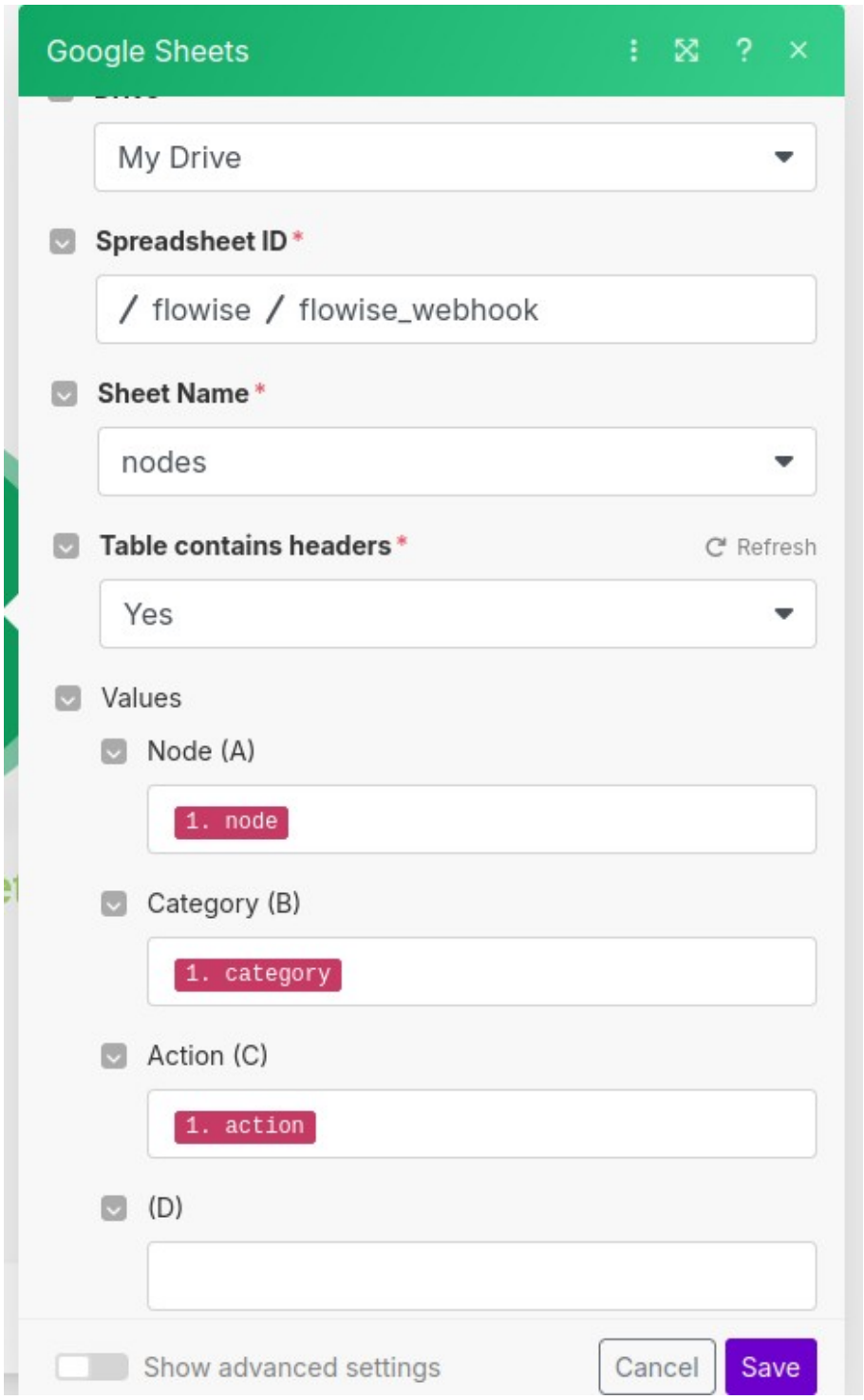


Figure 7: Begin mapping json values to gsheet's header.

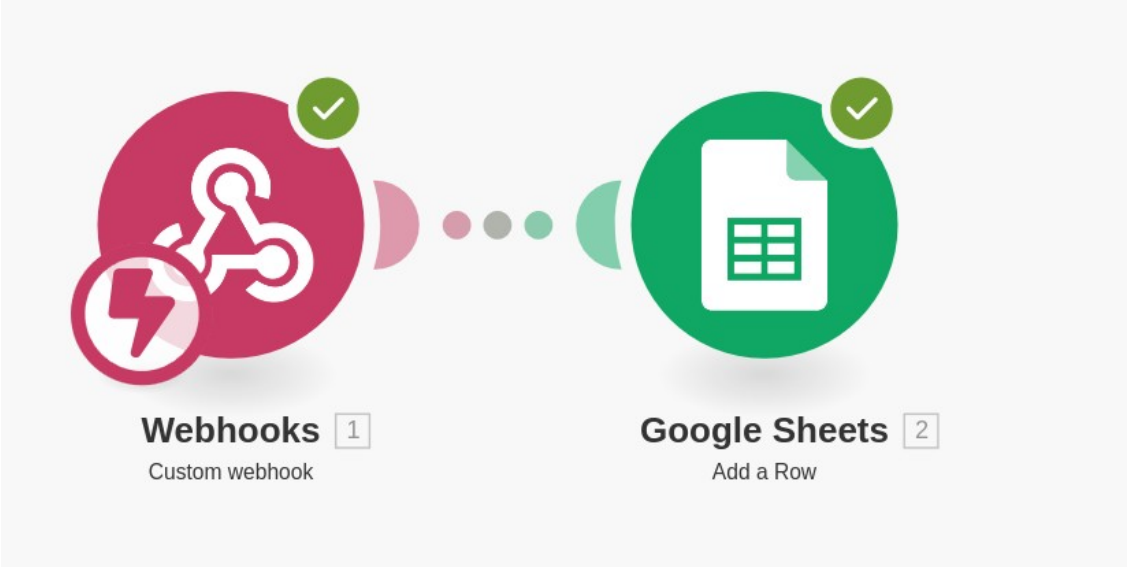


Figure 8: Once mapping is done, test the webhook by sending messages through postman.

9. Go to [Postman](#) and select: *Send an API request.*

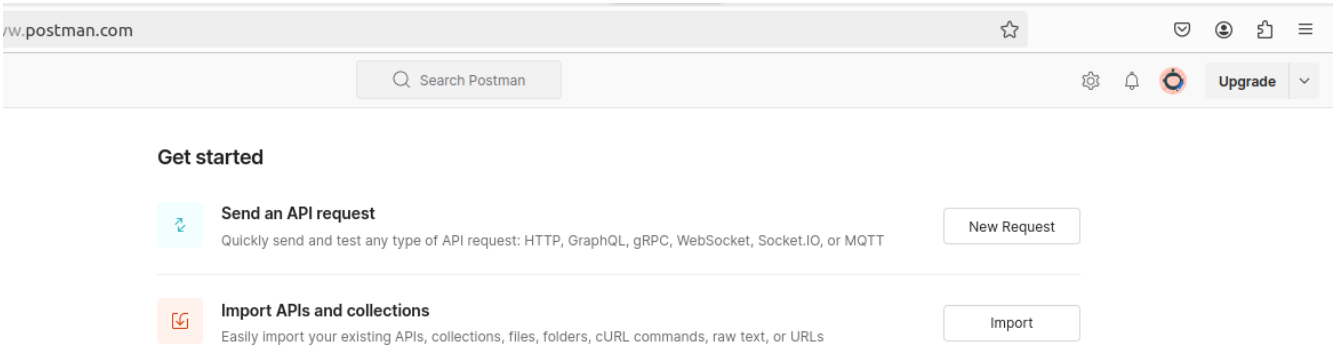


Figure 9: Click Send An API request in [Postman](#)

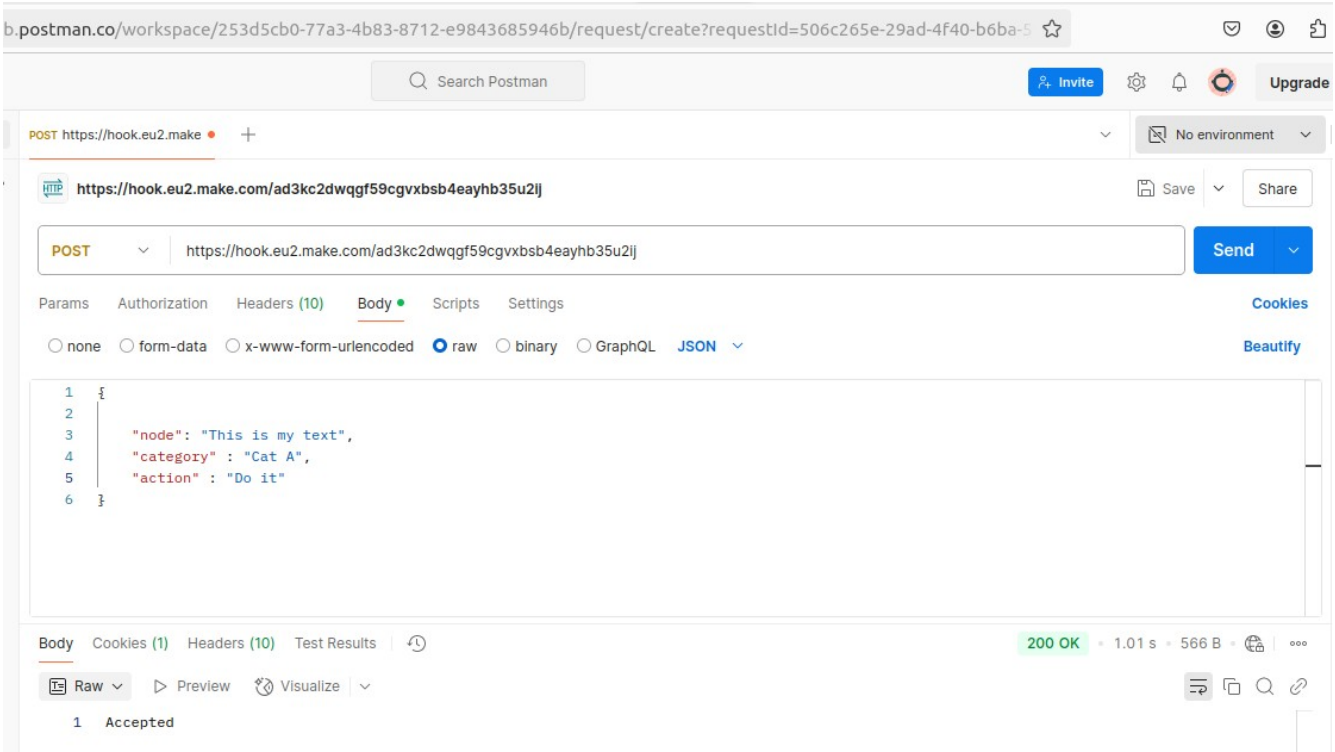


Figure 10: Creating a message. The JSON message has three keys, each corresonds to the header of gsheet. It MUST have the same names as the gsheet header.

10. Note the Green Colored sub-titles. Of names.

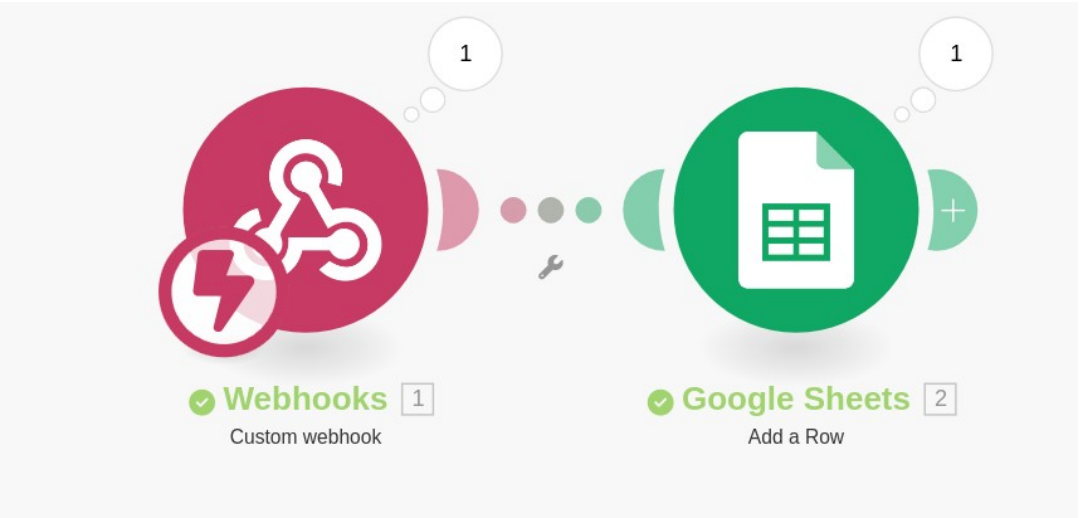
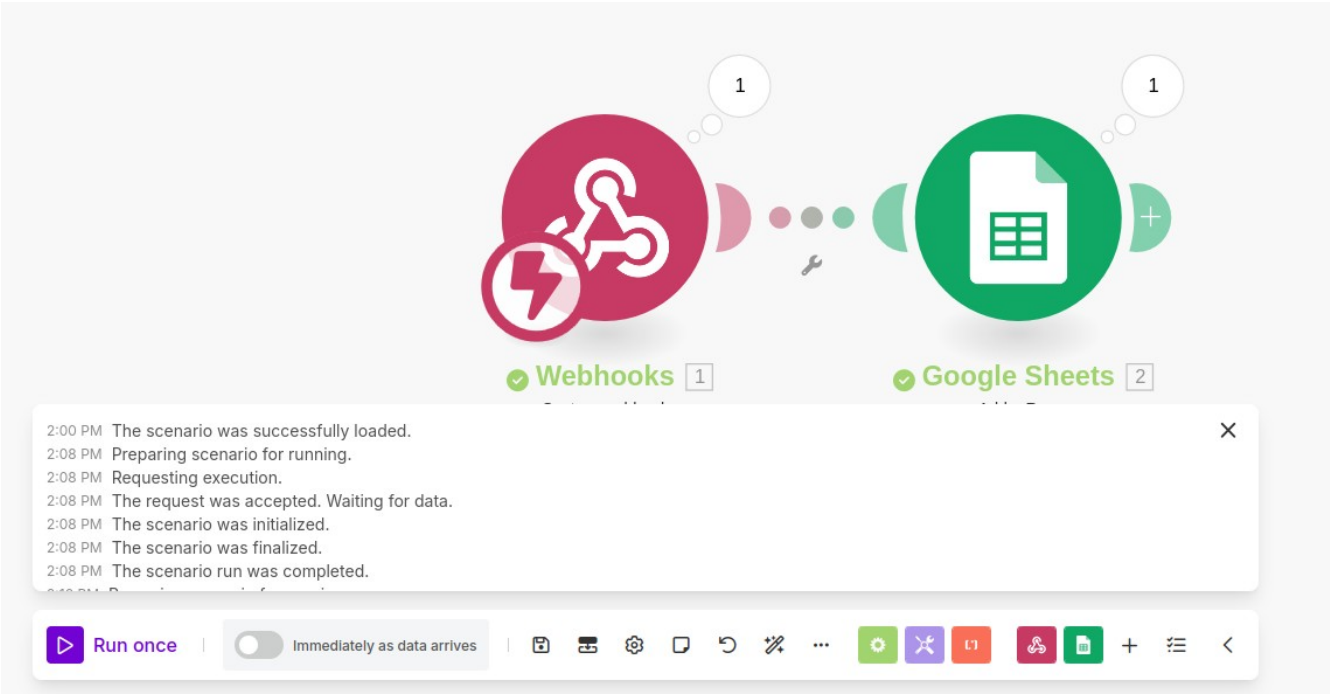
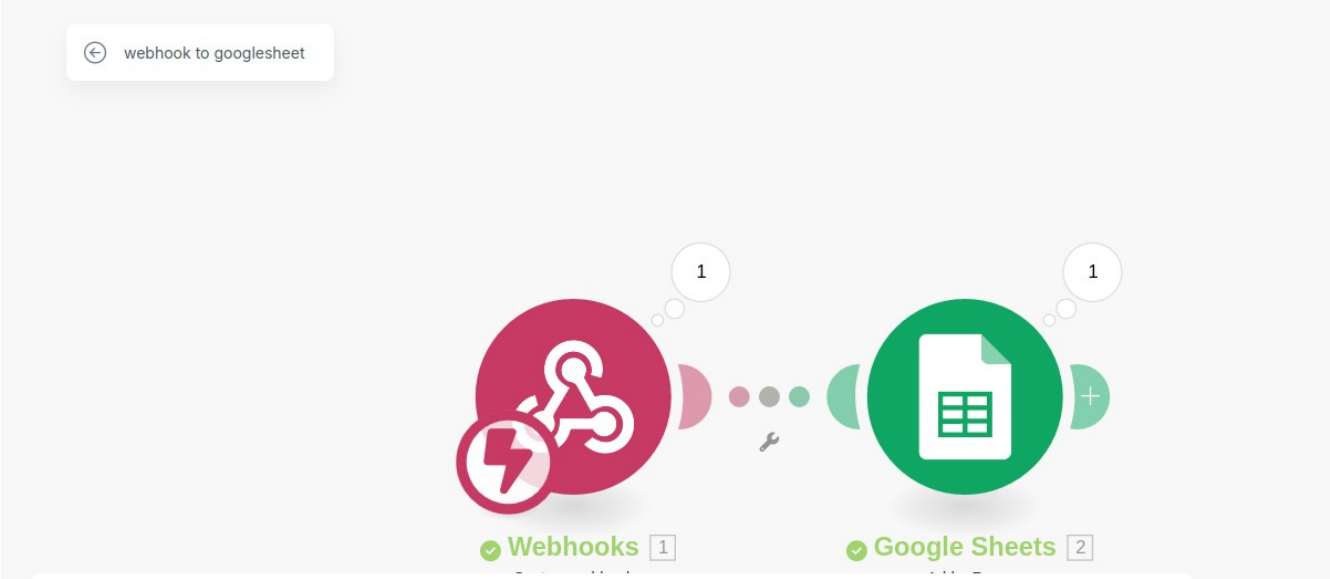


Figure 11: Message from Postman received into gsheet.

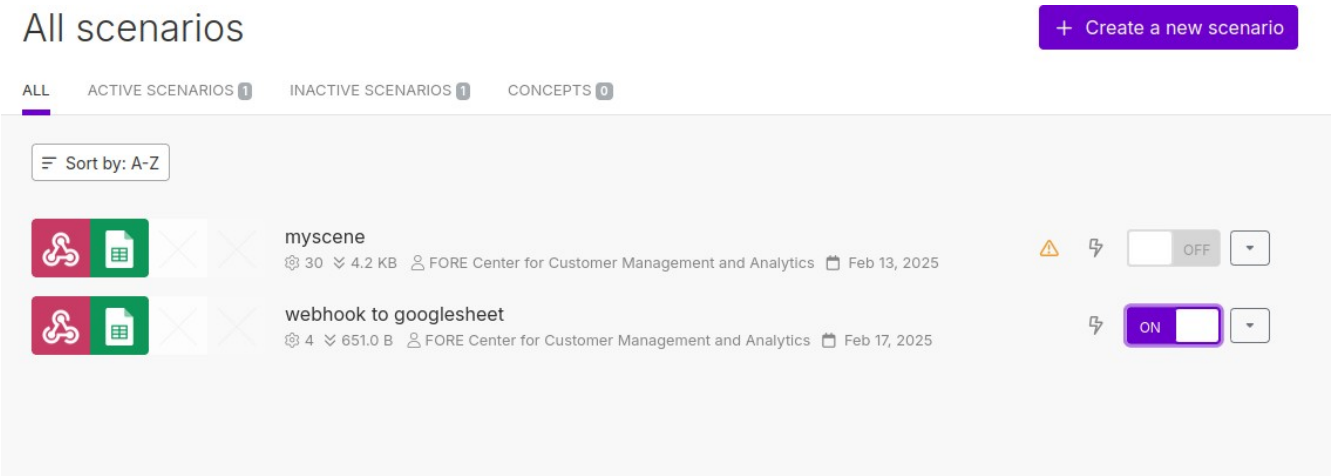
11. Check the logs in make.com by clicking on the last icon to the right of + or to the left of < at the bottom.



12. Change webhook description scenarion and save (top-right) below.



13. Switch your sceanrion to ON:



B. Workflow in Flowise:

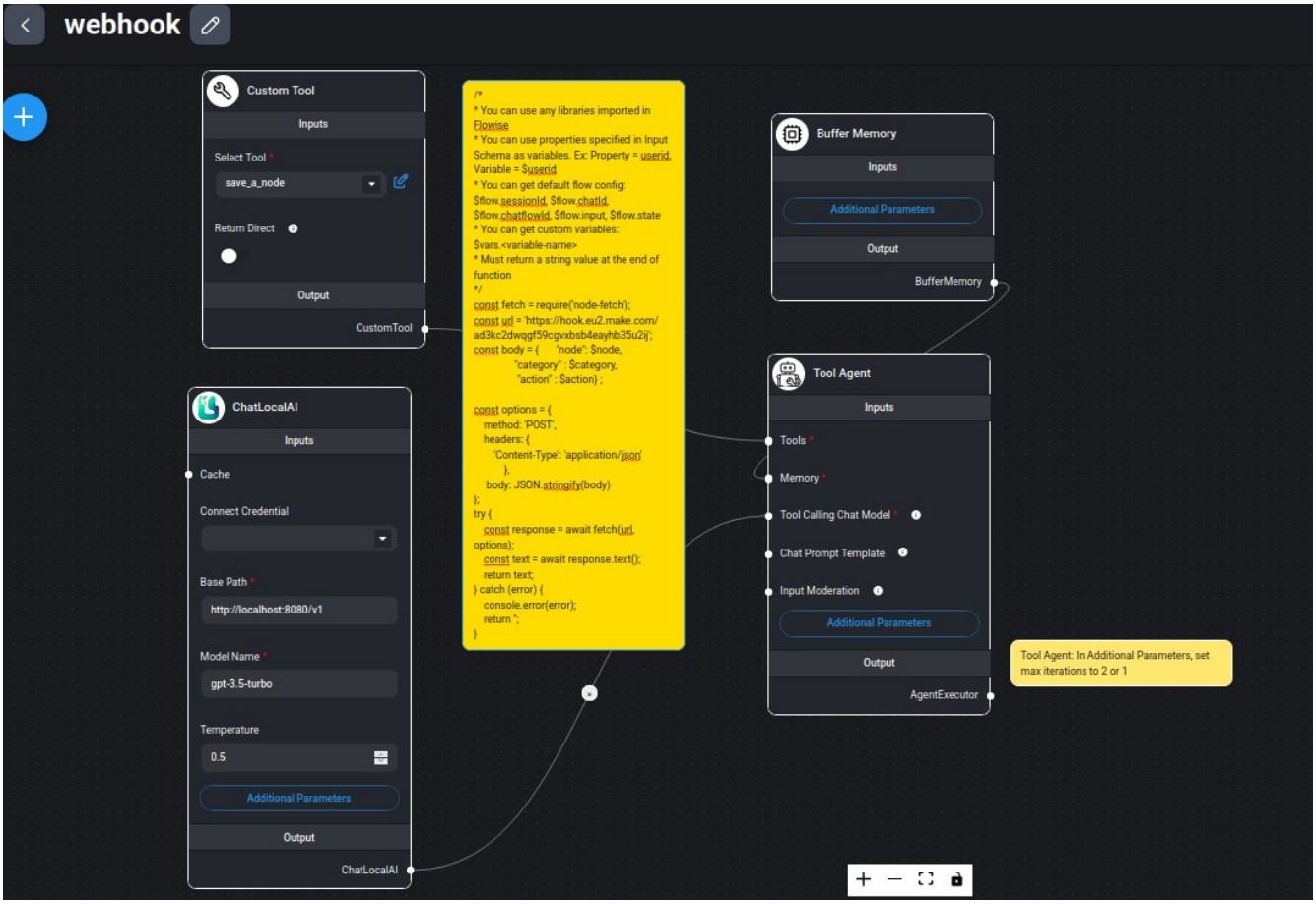


Figure 12: Complete flow in flowsie

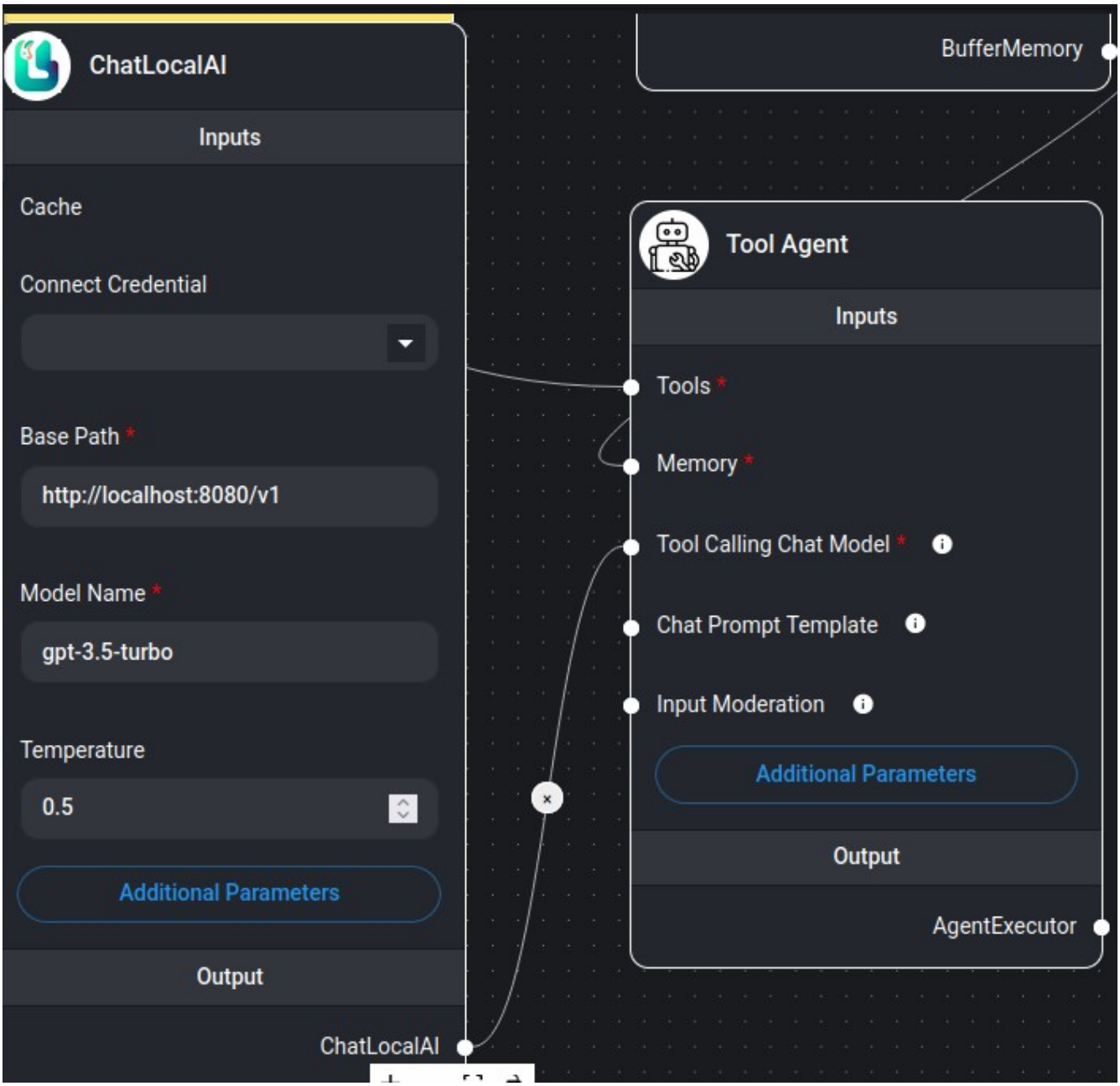


Figure 13: In Tool agent under Additional parameters, set max iteration to 1

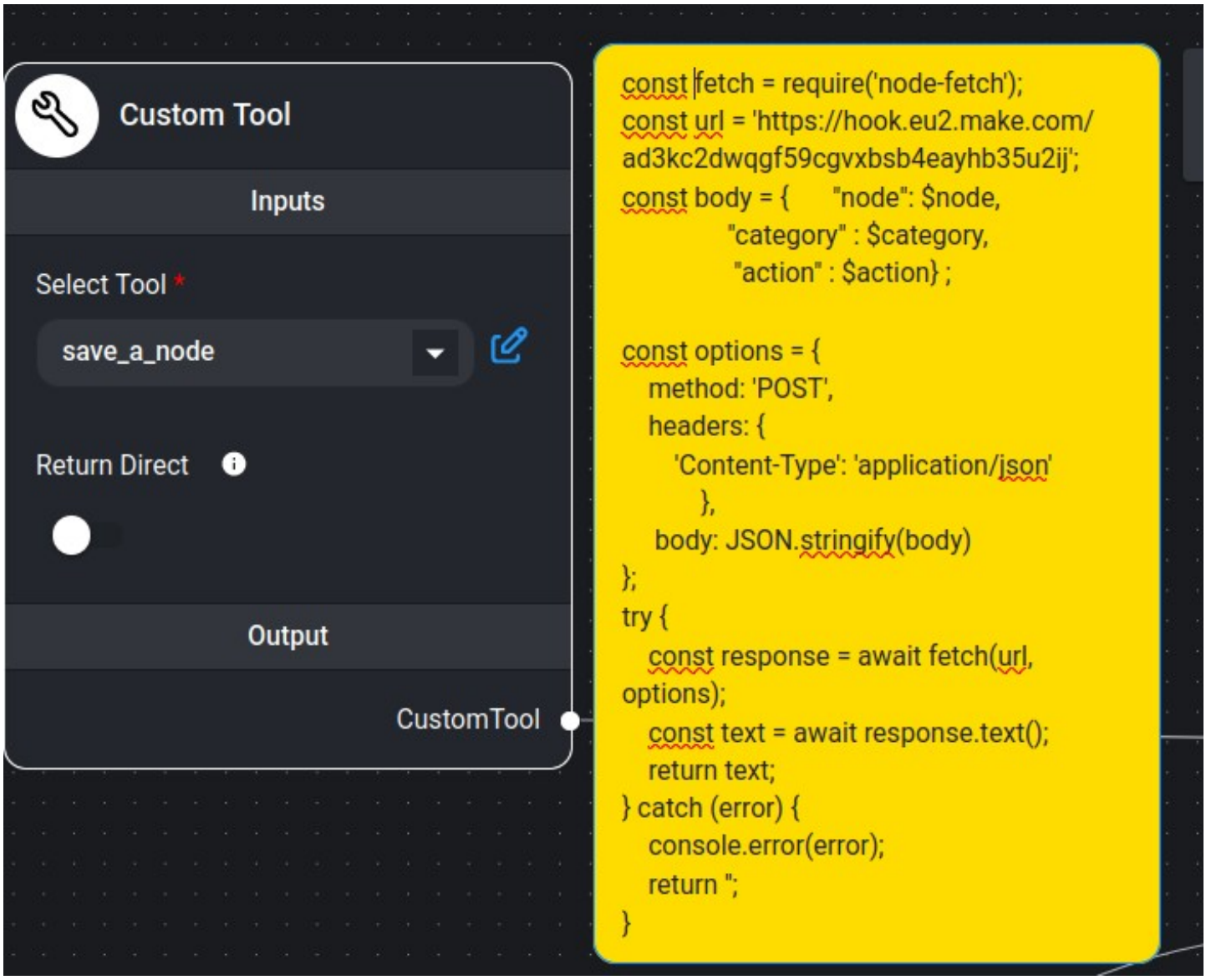


Figure 14: This is the complete code written in custom tool.

Edit Tool

Save As Template | Export

TOOL NAME *

Tool name must be small capital letter with underscore.
Ex: my_tool

save_a_node

TOOL DESCRIPTION *

Use this tool to save a node and optionally to write its description and action

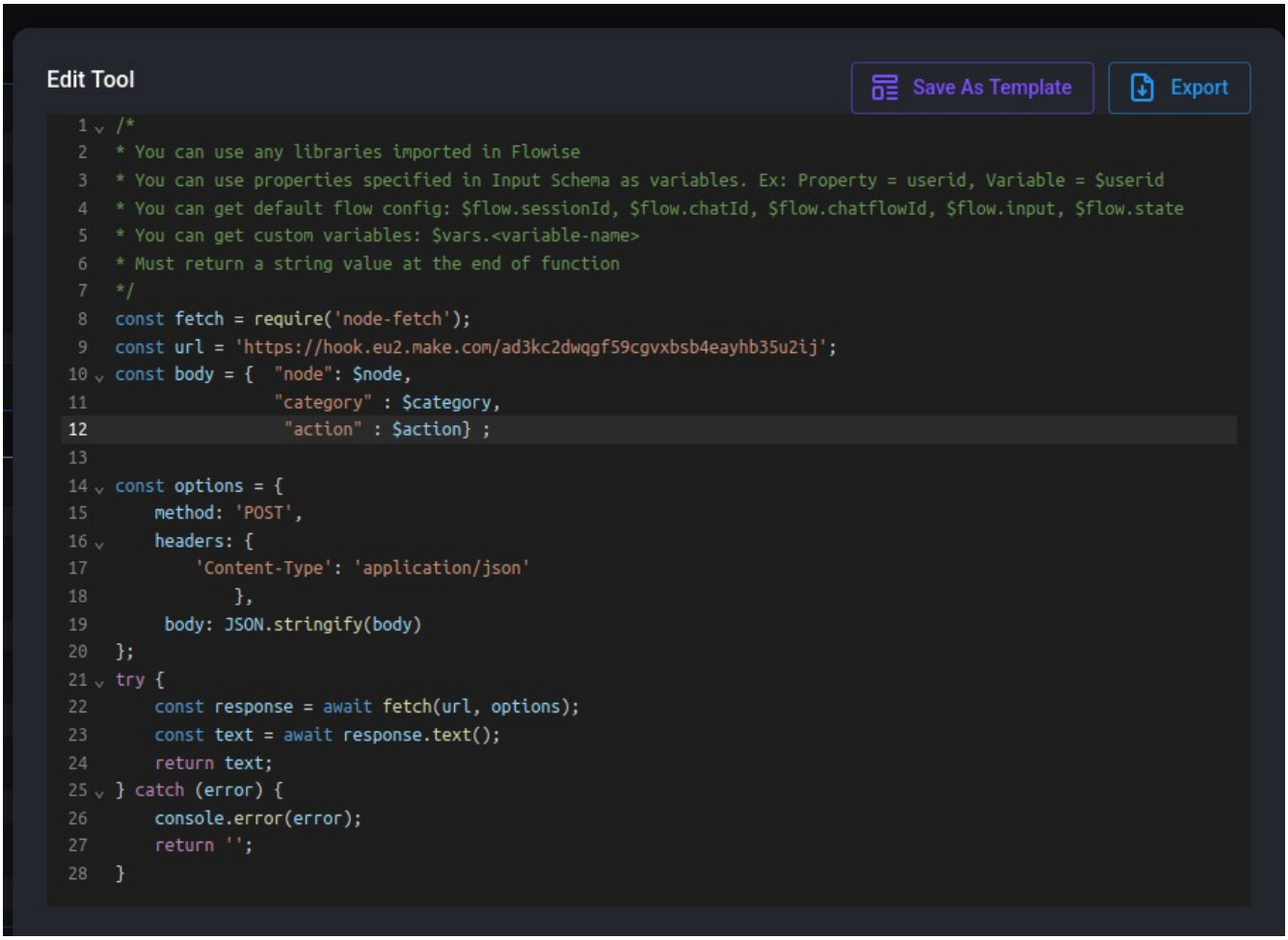
TOOL ICON SOURCE

https://raw.githubusercontent.com/gilbarbara/logos/main/logos/airtable.svg

INPUT SCHEMA

Add Item

Property	Type	Description	Required
node	string	The node to be saved	✓
category	string	Category of the node to be saved	✗



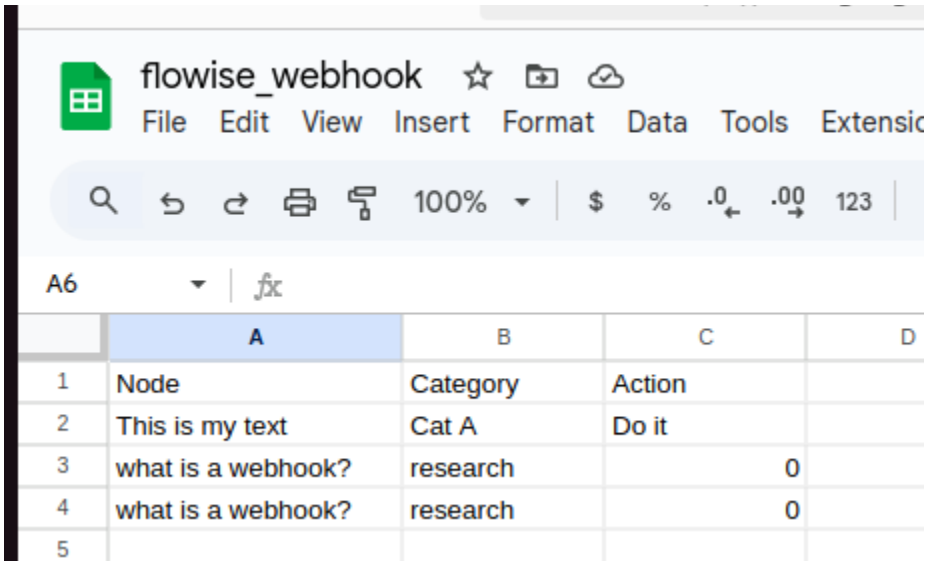
JS Code

```
/*
* You can use any libraries imported in Flowise
* You can use properties specified in Input Schema as variables. Ex: Property = userid, Variable =
$userid
* You can get default flow config: $flow.sessionId, $flow.chatId, $flow.chatflowId, $flow.input,
$flow.state
* You can get custom variables: $vars.<variable-name>
* Must return a string value at the end of function
*/
const fetch = require('node-fetch');

// Change as per your requirements below:
// url is the URL of the webhook
const url = 'https://hook.eu2.make.com/ad3kc2dwqgf59cgvxbsb4eayhb35u2ij';
const body = {   "node": $node,
                "category" : $category,
```

```
                "action" : $action} ;

// Do not change the code below
const options = {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json'
            },
     body: JSON.stringify(body)
};
try {
    const response = await fetch(url, options);
    const text = await response.text();
    return text;
} catch (error) {
    console.error(error);
    return '';
}
```



****